

Technical Test Assignment

Objective

Evaluate the candidate's ability to write clean, maintainable automated tests for Web UI and API.

Only **TypeScript with Playwright** is allowed, as it aligns with the automation stack used in our company.

Submit **both Web UI and API automation tests in the same project**. The assignment will be graded on the framework design and test coverage.

Submission Requirement (Important)

Please submit a GitHub repository link containing:

- All source code
- A README.md with instructions on how to run your script.
- Any configuration files, test data, or dependencies

Test Overview

1. Web UI Automation

Website to test: Swag labs (<https://www.saucedemo.com/>)

Scenario:

1. Navigate to the login page.
2. Log in with the following credentials:
 - a. Username: standard_user
 - b. Password: secret_sauce
3. Verify the products page loads.
4. Add any product to the cart.
5. Open the cart and verify if the correct item is listed.
6. Proceed checkout and fill in the required details.
7. Complete the purchase.

8. Validate that the success/confirmation message appears.

Requirements

- It will be a bonus of the test designed with page object model (POM)
- Test report generated after test execution.
- Use stable selectors.
- Include assertions.
- Provide clean, maintainable code and structure.
- Tests must be runnable from the command line.

2. API Automation

Objective: Use the public API at <https://jsonplaceholder.typicode.com/> and demonstrate your API testing skills.

Your Task: Automate API tests for the following CRUD operations on the /posts endpoint. **The key focus is on WHAT you choose to validate** - show us your testing thought process and what you consider important to verify.

Scenario:

- **CREATE** - POST a new post with your own data
 - Get the `id` from the response to use in subsequent steps
- **READ** - GET the post you just created using the `id` from step 1
 - Verify the data matches what you created
- **UPDATE** - PUT/PATCH the post to modify specific field(s)
 - Example: Change a word in the `body` or update the `title`
 - You decide what to modify
- **VERIFY UPDATE** - GET the post again to confirm changes were applied
 - Verify the updated fields changed correctly
 - Verify unchanged fields remained the same
- **DELETE** - DELETE the post using the same `id`
- **VERIFY DELETION** - Try to GET the deleted post

Requirements

- Use any API automation approach or library supported by your chosen framework.
- Tests must be runnable from the command line.
- **Give screenshot of console output and html test results**